

# Task 3: Introduction to FINN and PYNQ

Chair of Dependable Nano Computing, Karlsruhe Institute of Technology

Instructors: Vincent Meyers, Johannes Reibold

June 16, 2026

## 1 Introduction

In this task, you will first set up the device that will be used for inference, the PYNQ-Z2. The PYNQ-Z2 is a low-cost Zynq 7000 development board suitable for beginner and more advanced projects. It features a Zynq-7000 XC7Z020-1CLG400C SoC with Artix 7 Series FPGA and a dual core ARM Cortex-A9 CPU. This is one of the boards supported by the FINN compiler.

To load and test the neural network accelerators, we will be running the PYNQ environment (PetaLinux kernel) on the PYNQ-Z2, which provides the open-source Jupyter notebook infrastructure. The device can be accessed through a browser and programmed with python. You will find more details on this in [Section 2](#).

In PYNQ, programmable logic (PL / FPGA) circuits are presented as hardware libraries called *overlays*. These overlays, which can be accessed through an Python API on the processing system (PS), are analogous to software libraries. An overview of the interaction between PS, PL and peripherals is depicted in [Figure 1](#).

Please set up the device as described in [Section 2](#) during the lab, as you can not take it home!

## 2 Setting up the PYNQ-Z2

### 2.1 Board Setup

In the following setup instructions, numbers in parenthesis correspond to numbers in [Figure 2](#).

1. Set the **\*\* Boot\*\*** jumper to the SD position. (This sets the board to boot from the Micro-SD card)
2. To power the board from the micro USB cable, set the Power jumper to the USB position
3. Insert the Micro SD card loaded with the PYNQ-Z2 image into the Micro SD card slot underneath the board
4. Connect the USB cable to your PC/Laptop, and to the PROG - UART MicroUSB port on the board

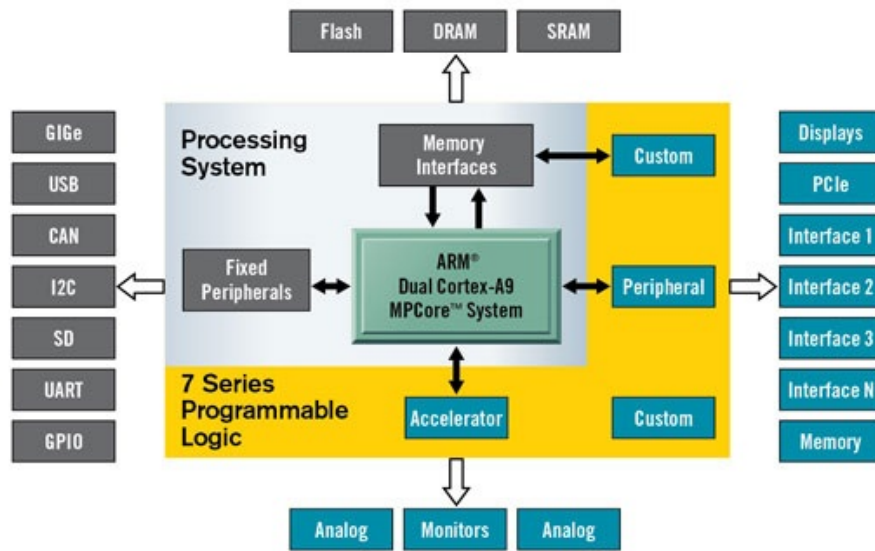


Figure 1: Overview of the Processing System and Programmable Logic

5. Connect the Ethernet port by following the instructions below

6. Turn on the PYNQ-Z2 and check the boot sequence by following the instructions below

As indicated in step 6, slide the power switch to the ON position to turn on the board. The Red LED will come on immediately to confirm that the board has power. After a few seconds, the Yellow/Green / Done LED will light up to show that the Zynq® device is operational.

After a minute you should see two Blue \*\* LEDs and four \*\*Yellow/Green LEDs flash simultaneously. The Blue LEDs will then turn on and off while the Yellow/Green LEDs remain on. The system is now booted and ready for use.

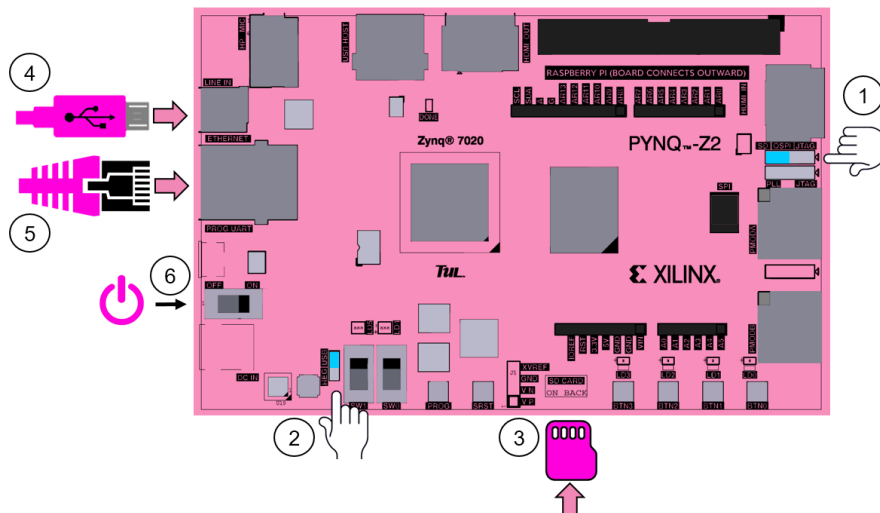


Figure 2

## 2.2 Network connection

Once your board is setup, you need to connect to it to start using Jupyter notebook. After connecting to the USB-Ethernet adapter on your lab PC, you need to find out the IP address of the board to access it. Use the following command and search for the IP address starting with 10.42.\*.\*: `arp -a`

Please come to us directly if you need any help setting this up!

### 2.2.1 Sharing a WiFi connection over Ethernet (OPTIONAL; NOT ON LAB PC)

You will need to have an Ethernet port available on your computer, and you will need to have permissions to configure your network interface. With a direct connection, you will be able to use PYNQ, but unless you can bridge the Ethernet connection to the board to an Internet connection on your computer, your board will not have Internet access. You will be unable to update or load new packages without Internet access. On Linux:

1. Connect the board to your computer's Ethernet port
2. Open NetworkManager (Settings > Network)
3. Choose the wired connection for editing
4. Go to the ipv4 settings tab there in the Method select *Shared to Other Computers*
5. Set your IP to 10.42.0.1 with the netmask 24
6. Open a terminal and type `ip address`. It should show the new connection with the correct IP and the devicename of the ethernet port (e.g. `enp3s0`)
7. Type `sudo arp-scan -l -I <devicename>` or `nmap -sn 10.42.0.1/24`
8. Browse to the ip address for the device
9. Open a terminal in the PYNQ environment and type `ping www.google.com`

### 2.2.2 Troubleshooting

If you have issues with sharing the internet connection with the PYNQ please try the following:

- `sysctl net.ipv4.ip_forward=1`
- `sudo iptables -P FORWARD ACCEPT`
- Inspect your firewall (disable and try again)

## 2.3 Neural Network Inference on the PYNQ-Z2

**Please do not modify or delete any files already on the SD card.** You can find some compiled bitfiles and hardware-handoff files in "Task-3-data/bitfiles". The files starting with "TFC" are 4 layer MLPs with 64 neurons in each layer and are trained on MNIST. Files starting with CNV are convolution neural network accelerators trained on the CIFAR-10 dataset. *w* describes the weight bit-width and *a* the activation bit width.

Follow the steps:

1. Open the jupyter interface in the browser with the ip address of the FPGA
2. Upload the `fpgannlab.zip` to the FPGA
3. Open a terminal through the jupyter interface and navigate to `/home/xilinx/jupyter_notebooks`
4. Run `unzip fpgannlab.zip`
5. Open the notebook `test_mnist.ipynb` in the extracted directory and click "Run all". Check the accuracy

Now, if everything works, please familiarize yourself with the contents of the notebook you just executed. Cell 1 instantiates the overlay with specified bitstream (path to the .bit file) and loads the train and test data. The overlay is defined in `driver.py`, which shows you how you can interact with the FPGA from python. Cell 2 sends data from the ARM to the FPGA, reads back the result and calculates the accuracy over the test set.

## 3 NN Accelerator Generation with FINN

Next, we will use FINN (specifically the `finn-plus` package) to compile our own bitstream for a given onnx file. You can find the onnx files in the folder "models" and the `build.py` to set the build configuration in Task-3-data. In this python file you define for which models and device the accelerator is generated. Additionally, you set the build configuration here. Please see [here](#) for more information on the attributes of the configuration. We already provide a working example, you only need to uncomment the model you want to compile the bitstream for.

In the introduction we presented how FINN uses folding to optimize and reduce the hardware resources required for implementing neural network layers on FPGAs. Folding balances hardware utilization and latency by grouping multiple neural network operations into a single hardware Processing Element (PE), enabling parallel computation. This maximizes hardware usage but can potentially increase latency if more folding is applied. FINN allows to control the PE and SIMD folding (or neuron and synapse fold) in a separate configuration file. It can also automatically choose a folding configuration for given target fps. We will explore both of those options in this task.

### 3.1 Configure FINN

1. Configure FINN using `finn deps update`. Use default configurations by pressing enter.
2. Update and configure FINN using `finn deps update` (yes, currently you need to run it twice).

### 3.2 Automatic Folding using Target FPS

Let's start by choosing the TFC-W1A1 model for compilation. Uncomment the required line in the python file. Then follow the steps in order:

1. In `build.yaml` specify the target fps to 100000 and the output directory as `output_auto`

2. Open a terminal and set up the conda and vivado environment (e.g. `source /Software/xilinx/Vivado/2022.2/settings64.sh` on the lab PC)
3. Run `finn build build.yaml`
4. Check in the reports (`estimate_network_performance` and `rtlsim_performance`) if target fps was reached

If you have access to the device (in the lab) you can also try your compiled bitfile on the device by following these steps:

1. In the outputs you should see a folder named `bitfile`. Rename the `.bit` and `.hwh` file inside to `tfc_w1a1_auto.bit` and `tfc_w1a1_auto.hwh`
2. Upload the files to the PYNQ-Z2 "bitfiles" folder
3. Open the `test_mnist.ipynb`
  - a) Change filename in the cell 1 to `tfc_w1a1_auto.bit`
  - b) In the directory `driver` generated by FINN, you can find the file `settings.json` with the `io_shape.dict` required for running the accelerator. You may need to adapt cell 1 in the notebook to match the shapes.
4. Run all and observe the outputs

FINN also created a folding configuration file that we can use as a starting point for a manual configuration. We will use this in the next step.

### 3.3 Manual Folding

The automatically created folding configuration is not always optimal due to FINN overestimating the required resources. Thus, you will now try to maximize the FPS (check reports) by maximizing resource utilization.

Follow these steps:

1. Copy the `auto_folding_config.json` to your project directory and rename it
2. Comment `target_fps` in `build.yaml`, uncomment `folding_config_file` and set it to the path of the file you copied
3. Adjust the number of PE and SIMD as you feel fit. The need to be devisors of the layer sizes of the model so that the layer can be mapped to the hardware with maximum utilization. The exact requirements are documented here: <https://finn.readthedocs.io/en/latest/internals.html#constraints-to-folding-factors-per-layer>.

To find out the model architecture, open the intermediate model from the last FINN run `step_target_fps_parallelization.onnx` in `netron.app` and make sure to show attributes with `Ctrl + D`

4. Run `finn build build.yaml`
5. Repeat until you maxed out resources (the compilation will fail, check the logs!)

Now check the throughput FPS and latency of your solution. Run a compilation with target fps again and set it to a higher value. Can FINN find a similar or better solution?

Feel free to play around with the other models.

## 4 Links

[https://pynq.readthedocs.io/en/latest/pynq\\_overlays/pynqz2.html](https://pynq.readthedocs.io/en/latest/pynq_overlays/pynqz2.html) [https://github.com/Xilinx/finn/blob/main/src/finn/builder/build\\_dataflow\\_config.py#L153](https://github.com/Xilinx/finn/blob/main/src/finn/builder/build_dataflow_config.py#L153)